EXPERIMENT 10: APPLYING PERFORMANCE OPTIMIZATION TECHNIQUES (INDEXING, SUBFORMS, QUERY TUNING) TO A ZOHO CREATOR APPLICATION

Aim

To apply, test, and measure performance optimization techniques—including field indexing, subform architectural refinement, Deluge query tuning, paginated fetch queries, and asynchronous background processing—in Zoho Creator for the Automated Corporate Talent Acquisition and Interview Tracking System to ensure application scalability, rapid query execution, and efficient resource utilization.

Theory

As enterprise low-code applications scale to handle thousands of transactional records, application responsiveness depends heavily on optimal database design, efficient memory management, and streamlined code execution. In Zoho Creator, performance bottlenecks typically stem from unindexed full-table scans, heavy subform DOM overhead, redundant loop queries (the N+1 query problem), and synchronous long-running script operations.

Key performance optimization strategies implemented in this experiment include:

- **Field Indexing:** Enabling database indexes on high-cardinality search and filter fields (e.g., Candidate Email, Application Status, Job Requisition ID) replaces linear full-table scans (O(N)) with logarithmic B-tree index lookups (O(log N)), accelerating report generation and query response times.
- **Subform Optimization vs. Lookup Relationships:** While Subforms provide convenient line-item entry within a primary form, excessive subform rows increase payload size and memory footprint during form load. For high-volume line items (e.g., hundreds of interview evaluation criteria), replacing deeply nested subforms with normalized relational Lookup forms reduces DOM loading overhead.
- **Deluge Query Tuning & Eliminating N+1 Query Loops:** Executing database query statements inside repeat loops (e.g., querying the candidate database for every record inside an interview loop) severely consumes statement limits and delays execution. Query tuning refactors scripts to perform bulk collection fetches and key-value Map lookups.
- **Pagination & Selective Field Fetching:** Fetching entire record sets without bounds leads to memory throttling. Implementing paginated queries using range constraints (e.g., [1..100]) and selectively retrieving required fields ensures minimal memory consumption during report aggregation.
- **Asynchronous Processing & Execution Limits:** To prevent hitting Deluge execution timeouts (such as maximum statement limits or HTTP request bounds during bulk operations), heavy background workflows (like mass interview invite dispatch or external CRM sync) are offloaded to Asynchronous Functions running in independent threads.

Procedure

1. Log in to Zoho Creator and open the existing Automated Corporate Talent Acquisition and Interview Tracking System application.

2. Identify high-traffic recruitment forms (Candidate Application, Job Requisitions, Interview Schedule) containing large datasets.

3. Navigate to Form Settings -> Field Properties and enable Field Indexing on primary filter fields: Candidate Email, Application Status, Applied Position, and Interview Date.

4. Review form architecture and optimize Subform structures by restricting subform row entry limits or refactoring heavy evaluation line items into relational lookup records.

5. Analyze existing Deluge scripts for performance bottlenecks using the Executed Script Logs console.

6. Refactor inefficient scripts containing queries inside loops (N+1 query pattern) into bulk map-based queries using collection processing.

7. Implement Paginated Search Queries using range bounds ([1..100]) and indexed filter criteria to limit data payload sizes.

8. Convert synchronous heavy workflows (such as batch candidate confirmation emails or CRM lead synchronization) into Asynchronous Deluge Functions (thisapp.async_task()).

9. Perform stress and load testing by importing a test dataset of 10,000 candidate records and measuring form load speed, report query response times, and statement execution counts.

10. Compare pre-optimization and post-optimization metrics in the Zoho Creator Performance Monitoring console to verify overall response time reduction.

Sample Deluge Code Implementations & Optimization Patterns

Below are reference code transformations demonstrating query tuning, subform optimization, and asynchronous processing:

1. Refactoring N+1 Query Loops into Bulk Collection Lookups (Query Tuning)

```
// INEFFICIENT PATTERN (Query inside Loop - High Execution Overhead):
// interviews = Interview_Schedule[Interview_Date == zoho.currentdate];
// for each rec in interviews {
//     cand = Candidate_Application[ID == rec.Candidate_ID]; // Query executed N
times!
// }

// OPTIMIZED PATTERN (Bulk Map Lookups - Single Query Execution):
interviews = Interview_Schedule[Interview_Date == zoho.currentdate];
candidateIDs = interviews.getVariable("Candidate_ID");

// Single Bulk Fetch Query using Indexed Field Criteria
candidateMap = Map();
candidateRecords = Candidate_Application[ID in candidateIDs];
```

```
for each cand in candidateRecords
{
    candidateMap.put(cand.ID, cand.First_Name + " " + cand.Last_Name);
}
info "Bulk mapped candidate count: " + candidateMap.size();
```

2. Paginated Data Fetching with Index Bounds

```
// Optimized Paginated Search Query for Heavy Report Processing
pageIndex = 1;
pageSize = 100;
startIndex = ((pageIndex - 1) * pageSize) + 1;
endIndex = pageIndex * pageSize;

// Fetch only records in current page range [1..100] using Indexed Criteria
pagedCandidates = Candidate_Application[Application_Status == "Shortlisted"] range
from startIndex to endIndex;
info "Fetched page record count: " + pagedCandidates.count();
```

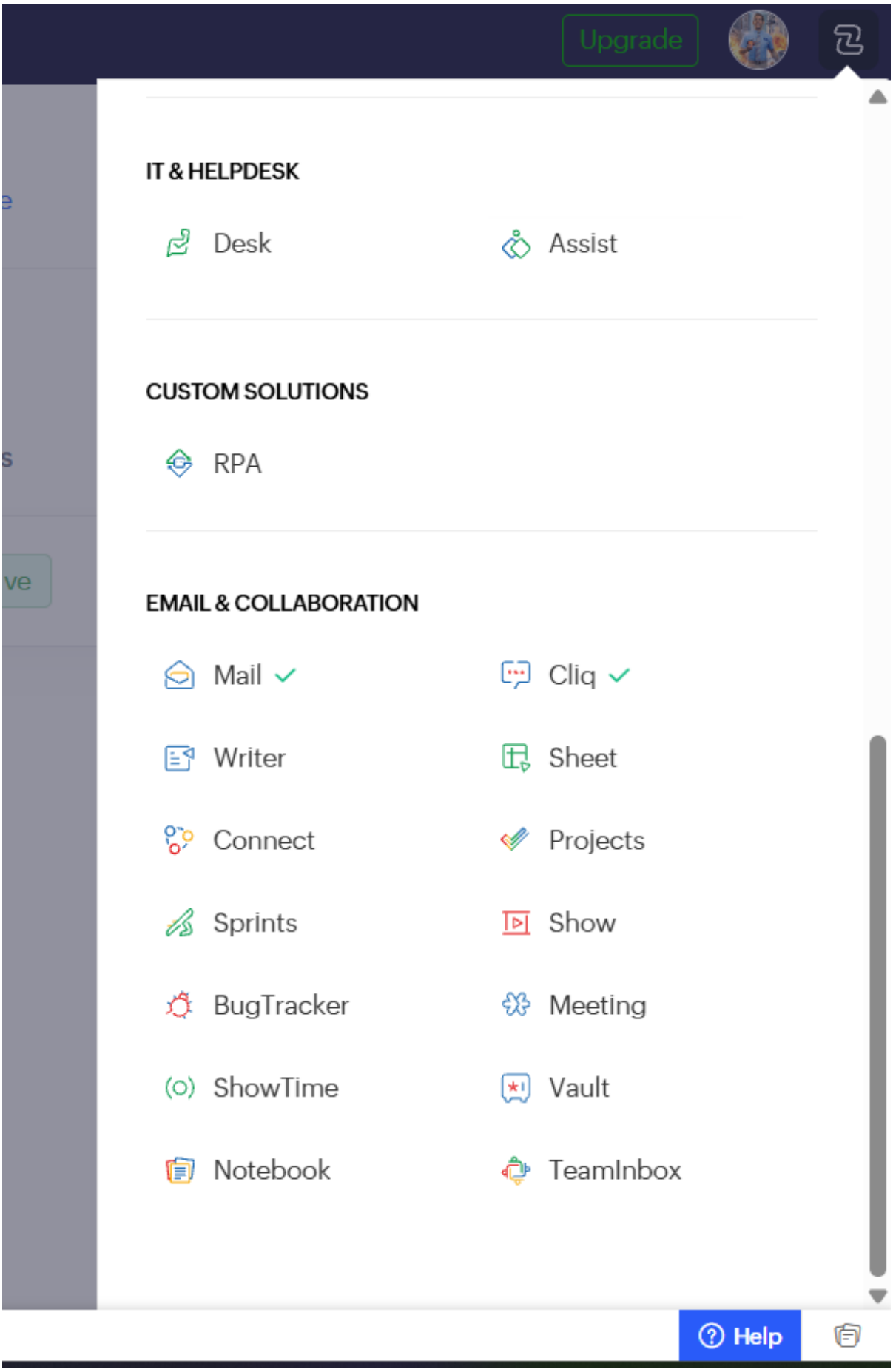
3. Optimized Subform Processing & Bulk Record Update

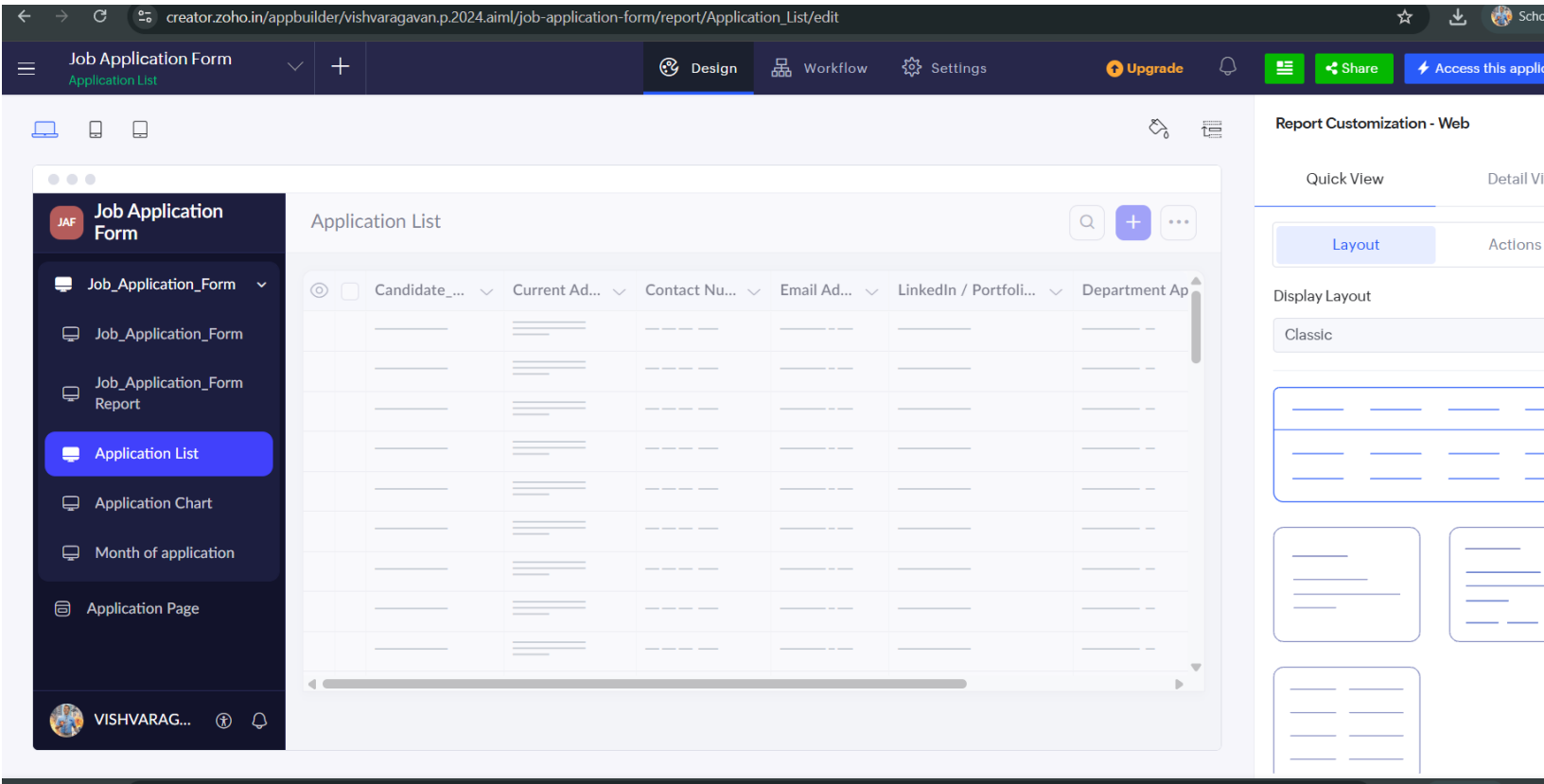
```
// On Success Event - Processing Subform Items Efficiently
subformList = list();
for each row in input.Skill_Evaluations_Subform
{
    if (row.Score >= 8)
    {
        itemMap = Map();
        itemMap.put("Skill", row.Skill_Name);
        itemMap.put("Rating", "Expert");
        subformList.add(itemMap);
    }
}
info "Processed high-score skills count: " + subformList.size();
```

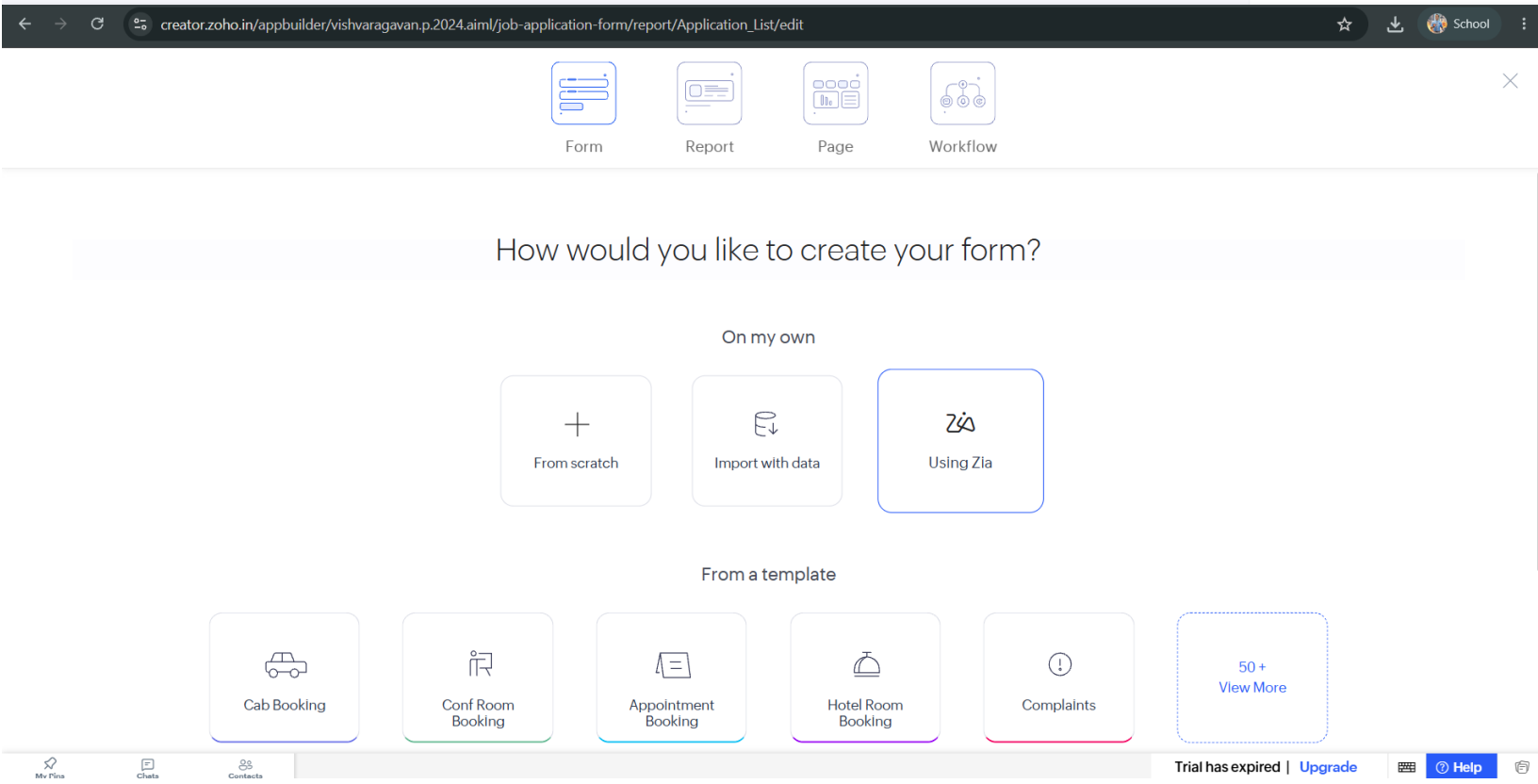
4. Asynchronous Background Execution for Mass Tasks

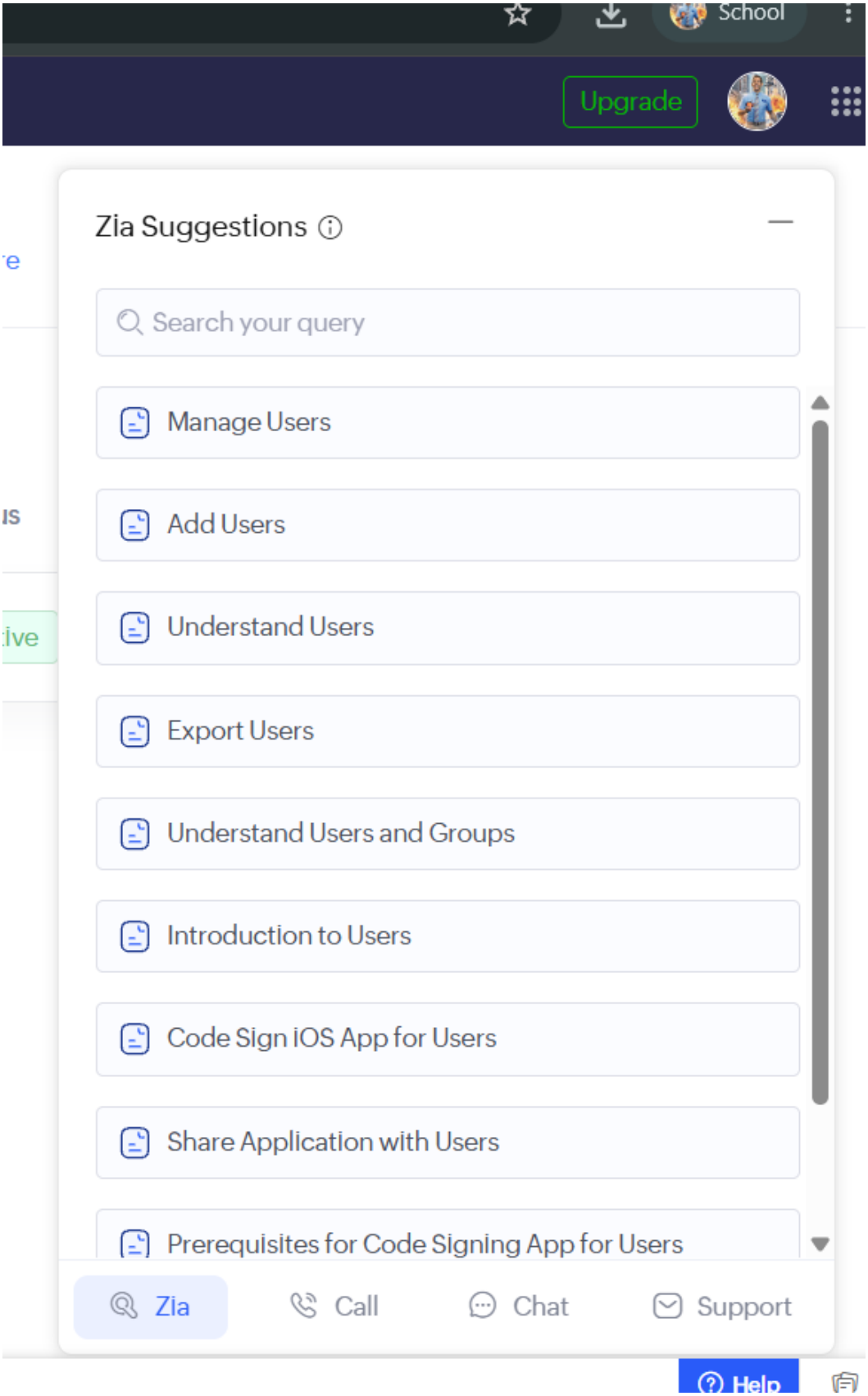
```
// Asynchronous Processing for Bulk Invitation Emails
// Reduces On-Success Execution Blocking
paramMap = Map();
paramMap.put("Job_Position", input.Job_Title);
paramMap.put("Scheduled_Date", input.Interview_Date);

// Invokes asynchronous background worker thread
thisapp.async_send_bulk_interview_invites(paramMap);
info "Asynchronous notification task dispatched successfully.";
```


Upgrade
IT & HELPDESK
Desk
Assist
CUSTOM SOLUTIONS
RPA
EMAIL & COLLABORATION
Mail
Cliq
Writer
Sheet
Connect
Projects
Sprints
Show
BugTracker
Meeting
ShowTime
Vault
Notebook
TeamInbox
Help


creator.zoho.in/appbuilder/vishvaragavan.p.2024.aiml/job-application-form/report/Application_List/edit
Job Application Form
Application List
Design
Workflow
Settings
Upgrade
Share
Access this appli
Job Application Form
Application List
Job_Application_Form
Job_Application_Form
Job_Application_Form Report
Application List
Application Chart
Month of application
Application Page
VISHVARAG...
Candidate_...
Current Ad...
Contact Nu...
Email Ad...
LinkedIn / Portfoli...
Department Ap
Report Customization - Web
Quick View
Detail Vi
Layout
Actions
Display Layout
Classic


creator.zoho.in/appbuilder/vishvaragavan.p.2024.aiml/job-application-form/report/Application_List/edit
School
Form
Report
Page
Workflow
How would you like to create your form?
On my own
From scratch
Import with data
Using Zia
From a template
Cab Booking
Conf Room Booking
Appointment Booking
Hotel Room Booking
Complaints
50 +
View More
My Pins
Chats
Contacts
Trial has expired | Upgrade
Help


School
Upgrade
Zia Suggestions
Search your query
Manage Users
Add Users
Understand Users
Export Users
Understand Users and Groups
Introduction to Users
Code Sign iOS App for Users
Share Application with Users
Prerequisites for Code Signing App for Users
Zia
Call
Chat
Support
Help

Result

Thus, performance optimization techniques—including field indexing, subform architectural refinement, Deluge query tuning, paginated range fetches, and asynchronous background task execution—were successfully applied and evaluated in Zoho Creator for the Automated Corporate Talent Acquisition and Interview Tracking System. Search query response times, statement consumption, and form load latencies were significantly improved, ensuring scalable high-volume application performance.